treats both operands as signed. `mulhsu` (*multiply high signed unsigned*) treats the first operand as signed and the second as unsigned, and `mulhu` (*multiply high unsigned unsigned*) treats both operands as unsigned. For example, `mulhsu t1, t2, t3` treats `t2` as a 32-bit signed (two's complement) number and `t3` as a 32-bit unsigned number, multiplies these two source operands, and puts the upper 32 bits of the result in `t1`. Using a series of two instructions—one of the "multiply high" instructions followed by the `mul` instruction—will place the entire 64-bit result of the 32-bit multiplication in the two registers designated by the user. For example, the following code multiplies 32-bit signed numbers in `s3` and `s5` and places the 64-bit product in `t1` and `t2`. That is, {`t1`, `t2`} = `s3` × `s5`.

```
mulh t1, s3, s5
mul  t2, s3, s5
```

6.3.3 Branching

Programs would be boring if they could only run in the same order every time, independent of the input. An advantage of a computer over a calculator is its ability to make decisions. A computer performs different tasks depending on the input. For example, *if/else* statements, *switch/case* statements, *while* loops, and *for* loops all conditionally execute code depending on some test. *Branch* instructions modify the flow of the program so that the processor can fetch instructions that are not in sequential order in memory. They modify the PC to skip over sections of code or to repeat previous code. *Conditional branch* instructions perform a test and branch only if the test is TRUE. Unconditional branch instructions, called *jumps*, always branch.

Conditional Branches

The RISC-V instruction set has six conditional branch instructions, each of which take two source registers and a label indicating where to go. `beq` (*branch if equal*) branches when the values in the two source registers are equal. `bne` (*branch if not equal*) branches when they are unequal. `blt` (*branch if less than*) branches when the value in the first source register is less than the value in the second, and `bge` (*branch if greater than or equal to*) branches when the first is greater than or equal to the second. `blt` and `bge` treat the operands as signed numbers, while `bltu` and `bgeu` treat the operands as unsigned.

There is no need for `bgt` or `ble` because these can be obtained by switching the order of the source registers of `blt` and `bge`. However, these are available as pseudoinstructions (see Section 6.3.8).

Code Example 6.12 illustrates the use of `beq`. When the program reaches the branch if equal instruction (`beq`), the value in `s0` is equal to the value in `s1`, so the branch is taken. Thus, the next instruction executed is the `add` instruction just after the label called `target`. The `addi` and `sub` instructions between the branch and the label are not executed.

Code Example 6.12 CONDITIONAL BRANCHING USING `beq`

RISC-V Assembly Code

```
  addi  s0, zero, 4       # s0 = 0 + 4 = 4
  addi  s1, zero, 1       # s1 = 0 + 1 = 1
  slli  s1, s1, 2         # s1 = 1 << 2 = 4
  beq   s0, s1, target    # s0 == s1, so branch is taken
  addi  s1, s1, 1         # not executed
  sub   s1, s1, s0        # not executed
target:                   # label
  add   s1, s1, s0        # s1 = 4 + 4 = 8
```

Assembly code uses *labels* to indicate instruction locations in the program. A label refers to the instruction just after the label. When the assembly code is translated into machine code, these labels correspond to instruction addresses (as will be discussed in Sections 6.4.3 and 6.4.4). RISC-V assembly labels are followed by a colon (:). Most programmers indent their instructions but not the labels to help make labels stand out.

In Code Example 6.13, the branch is not taken because `s0` is equal to `s1`, and the code continues to execute directly after the `bne` (branch if not equal) instruction. All instructions in this code snippet are executed.

Code Example 6.13 CONDITIONAL BRANCHING USING `bne`

RISC-V Assembly Code

```
  addi  s0, zero, 4       # s0 = 0 + 4 = 4
  addi  s1, zero, 1       # s1 = 0 + 1 = 1
  slli  s1, s1, 2         # s1 = 1 << 2 = 4
  bne   s0, s1, target    # branch not taken
  addi  s1, s1, 1         # s1 = 4 + 1 = 5
  sub   s1, s1, s0        # s1 = 5 – 4 = 1
target:
  add   s1, s1, s0        # s1 = 1 + 4 = 5
```

Jumps

A program can jump—that is, unconditionally branch—using one of three instructions: *jump* (`j`), *jump and link* (`jal`), or *jump register* (`jr`). `j` jumps directly to the instruction at the specified label. Code Example 6.14 shows the use of the `j` (jump) instruction to skip over three instructions and continue at the `add` instruction after the label `target`. After the `j` instruction executes, this program unconditionally continues executing the `add` instruction at the label `target`. All of the instructions between the jump and the label are skipped. We will discuss `jal` and `jr` instructions in Section 6.3.7, where they are used for function calls.

Code Example 6.14 UNCONDITIONAL BRANCHING USING `j`

RISC-V Assembly Code

```
  j    target        # jump to target
  srai s1, s1, 2     # not executed
  addi s1, s1, 1     # not executed
  sub  s1, s1, s0    # not executed
target:
  add  s1, s1, s0    # s1 = s1 + s0
```

6.3.4 Conditional Statements

If, if/else, and switch/case statements are conditional statements commonly used in high-level languages. They each conditionally execute a *block* of code consisting of one or more statements. This section shows how to translate these high-level constructs into RISC-V assembly language.

If Statements

An *if* statement executes a block of code, the *if block*, only when a condition is met. Code Example 6.15 shows how to translate an if statement into RISC-V assembly code. The assembly code for the if statement tests the opposite condition of the one in the high-level code. In Code Example 6.15, the high-level code tests for `apples == oranges`. The assembly code tests for `apples != oranges` using `bne` to skip the if block if the condition is not satisfied. Otherwise (i.e., when `apples == oranges`), the branch is not taken, and the if block is executed.

In C and many other high-level programming languages, the double equals sign, ==, is an equality comparison, returning TRUE if both sides are equal. != is an inequality comparison.

Code Example 6.15 IF STATEMENT

High-Level Code

```
if (apples == oranges)
  f = g + h;
apples = oranges - h;
```

RISC-V Assembly Code

```
# s0 = apples, s1 = oranges
# s2 = f, s3 = g, s4 = h
    bne s0, s1, L1 # skip if (apples != oranges)
    add s2, s3, s4 # f = g + h
L1: sub s0, s1, s4 # apples = oranges - h
```

If/else Statements

If/else statements execute one of two blocks of code, depending on a condition. When the condition in the if statement is met, the *if* block is executed. Otherwise, the *else* block is executed. Code Example 6.16 shows an example if/else statement.

Like if statements, if/else assembly code tests the opposite condition of the one in the high-level code. In Code Example 6.16, the high-level code tests for (`apples == oranges`) and the assembly code tests for

Code Example 6.16 IF/ELSE STATEMENT

High-Level Code

```
if (apples == oranges)
  f = g + h;
else
  apples = oranges - h;
```

RISC-V Assembly Code

```
# s0 = apples, s1 = oranges
# s2 = f, s3 = g, s4 = h
    bne s0, s1, L1 # skip if (apples != oranges)
    add s2, s3, s4 # f = g + h
    j   L2
L1: sub s0, s1, s4 # apples = oranges - h
L2:
```

(`apples != oranges`). If that opposite condition is TRUE, `bne` skips the if block and executes the else block. Otherwise, the if block executes and finishes with a jump (`j`) past the else block.

Switch/case Statements*

Switch/case statements, also called simply *case* statements, execute one of several blocks of code, depending on the conditions. If no conditions are met, the *default* block is executed. A case statement is equivalent to a series of nested if/else statements. Code Example 6.17 shows two high-level code snippets with the same functionality: they calculate whether to dispense $20, $50, or $100 from an ATM (automatic

Code Example 6.17 SWITCH/CASE STATEMENTS

High-Level Code

```
switch (button) {
  case 1:  amt = 20; break;

  case 2:  amt = 50; break;

  case 3:  amt = 100; break;

  default: amt = 0;
}

// equivalent function using
// if/else statements
if      (button == 1)  amt = 20;
else if (button == 2)  amt = 50;
else if (button == 3)  amt = 100;
else                   amt = 0;
```

RISC-V Assembly Code

```
# s0 = button, s1 = amt

case1:
  addi  t0, zero, 1       # t0 = 1
  bne   s0, t0, case2     # button == 1?
  addi  s1, zero, 20      # if yes, amt = 20
  j     done              # break out of case
case2:
  addi  t0, zero, 2       # t0 = 2
  bne   s0, t0, case3     # button == 2?
  addi  s1, zero, 50      # if yes, amt = 50
  j     done              # break out of case
case3:
  addi  t0, zero, 3       # t0 = 3
  bne   s0, t0, default   # button == 3?
  addi  s1, zero, 100     # if yes, amt = 100
  j     done              # break out of case
default:
  add   s1, zero, zero    # amt=0
done:
```

teller machine) depending on the button pressed. The RISC-V assembly implementation is the same for both high-level code snippets.

6.3.5 Getting Loopy

Loops repeatedly execute a block of code, depending on a condition. While loops and for loops are commonly used in high-level languages. This section shows how to translate them into RISC-V assembly language, taking advantage of conditional branching.

While Loops

While loops repeatedly execute a block of code while a condition is met—that is, *until* a condition is *not* met. The while loop in Code Example 6.18 determines the value of `x` such that $2^x = 128$. It executes seven times, until `pow` = 128.

Code Example 6.18 WHILE LOOP

High-Level Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;


while (pow != 128) {
  pow = pow * 2;
  x = x + 1;
}
```

RISC-V Assembly Code

```
#  s0 = pow, s1 = x
       addi s0,  zero, 1      # pow = 1
       add  s1,  zero, zero   # x = 0

       addi t0,  zero, 128    # t0 = 128
while: beq  s0,  t0, done     # pow = 128?
       slli s0,  s0, 1        # pow = pow * 2
       addi s1,  s1, 1        # x = x + 1
       j    while             # repeat loop
done:
```

Like if/else statements, the assembly code for while loops tests the opposite condition of the one in the high-level code. If that opposite condition is TRUE (in this case, `s0` == 128), the while loop is finished. Otherwise, the branch isn't taken and the loop body executes. Code Example 6.18 initializes `pow` to 1 and `x` to 0 before the while loop. The while loop compares `pow` to 128 and exits the loop if it is equal. Otherwise, it doubles `pow` (using a left shift), increments `x`, and branches back to the start of the while loop.

Do/while loops are similar to while loops, but they execute the body of the loop once before checking the condition. Code Example 6.19 illustrates such a loop. Notice that, unlike previous examples, the branch checks the same condition as in the high-level code.

For Loops

It is very common to initialize a variable before a while loop, check that variable in the loop condition, and change that variable each time

Code Example 6.19 DO/WHILE LOOP

High-Level Code

```
// determines the power
// of x such that 2^x = 128
int pow = 1;
int x   = 0;

do {
  pow = pow * 2;
  x = x + 1;
} while (pow != 128);
```

RISC-V Assembly Code

```
#  s0 = pow, s1 = x
       addi s0,  zero, 1       # pow = 1
       add  s1,  zero, zero    # x = 0

       addi t0,  zero, 128     # t0 = 128
while: slli s0,  s0, 1         # pow = pow * 2
       addi s1,  s1, 1         # x = x + 1
       bne  s0,  t0, while     # pow = 128?
done:
```

through the while loop. *For* loops are a convenient shorthand that combines the initialization, condition check, and variable change in one place. The high-level code format of the for loop is:

```
for (initialization; condition; loop operation)
  statement
```

The initialization code executes *before* the for loop begins. The condition is tested at the *beginning of each* loop iteration. If the condition is not met, the loop exits. If the condition is met, the statement (or statements) in the loop body are executed. The loop operation executes at the *end* of each loop iteration.

Code Example 6.20 adds the numbers from 0 to 9. The loop variable, in this case `i`, is initialized to 0 and is incremented at the end of each loop iteration. The for loop executes as long as `i` is less than 10. Note that this example also illustrates relative comparisons. The loop checks the < condition to continue, so the assembly code checks the opposite condition, >=, to exit the loop.

For loops are especially useful for accessing large amounts of similar data stored in memory arrays, which are discussed next.

Code Example 6.20 FOR LOOP

High-Level Code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i = 0; i < 10; i = i + 1) {
   sum = sum + i;
}
```

RISC-V Assembly Code

```
# s0 = i, s1 = sum
      addi s1, zero, 0      # sum = 0
      addi s0, zero, 0      # i = 0
      addi t0, zero, 10     # t0 = 10
for:  bge  s0, t0, done     # i >= 10?
      add  s1, s1, s0       # sum = sum + i
      addi s0, s0, 1        # i = i + 1
      j    for              # repeat loop
done:
```